Ex.No:12a) 12. End - end communication at Transport layer.

Aim:

Implement echo client using TCP/UDP sockets.

Algorithm:

```
import socket
import threading
def start_server():
  server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind(('127.0.0.1', 5000))
server.listen(1)
print("[server] listening on 127.0.0.1:5000")
conn, addr = server.accept()
print(f"[server] connected by {addr}")
while True:
  data = conn.recv(1024).decode()
  if not data or data.lower() == 'quit':
   print("[Server] Connection closed by client.")
   break
   print(f"[Server] received: {data}")
   conn.send(f"Server received: {data}".encode())
conn.close()
server.close()
if __name__ == "__main__":
server_thread = threading.thread(target = start_
            server, daemon = True)
 server_thread.start()
 start_client().
```

Output:

[server] listening on 127.0.0.1:5000 [client] connected to server

[server] connected by ('127.0.0.1', 53458) Enter message (or 'quit' to exit):

hello.

[Server] Received: hello.

[Server Response] Server received: hello.

Ex.No:12b) End-end communication at Transport layer.

Aim:

Implement chat client server using TCP/UDP packet sockets.

Algorithm:

```
import socket
import threading
def udp_server():
    s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM
    s.bind(('127.0.0.1', 12000))
    print("[Server] running on 127.0.0.1:12000")
    while True:
    data, addr = s.recvfrom(1024)
    msg = data.decode()
    if msg.lower() == 'quit':
    print(f"[Server] client {addr} ended
                the connection.")
     break
     print(f"[Server] received from {addr}: {msg}")
     s.sendto(data, addr)
     s.close()
     print("[Server] stopped.")
def udp_client():
    c = socket.socket(socket.AF_INET,
                socket.SOCK_DGRAM)
    server = ('127.0.0.1', 12000)
    while TRUE:
    msg = input("Enter message (or 'quit' to stop):")
    c.sendto(msg.encode(), server)
    if msg.lower() == 'quit':
        print("[CLIENT] closing connection.....")break
    data, _ = c.recvfrom(1024)
```

Scanned with OKEN Scanner

```
s.close()
print("[Server] Stopped.")
def udp-client():
 c=socket.socket(socket,AF-INET, saket, sock-DGRAM)
 server=('127.0.0.1', 12000)
 while True:
  msg=input("Enter message (or 'quit' to stop):")
  c.sendto(msg.encode(), server)
  if msg.lower()=='quit':
   print("[client] closing connection....") break
  data,-=c.recvfrom(1024)
  print("Echo from Server:", data.decode())
  c.close()
  print("[client] Stopped.")
  if -name-=="-main-":
   threading.thread(target=udp-server,
                    daemon=True).start()
   udp-client().
```

Output:

Enter message (or 'quit' to stop): [Server] running on 127.0.0.1:12000 hi.

[Server] Received from ('127.0.0.1', 49670): hi

Echo from Server: hi

Enter message (or 'quit' to stop): hello

[Server] received from ('127.0.0.1', 49670): hello

Echo from server: hello.

Scanned with OKEN Scanner